Merakié

Plan de trabajo para convertir el prototipo en una aplicación web publicada

Nicole Julay Ramirez Silva & Santiago Muñoz Eslava · Proyecto POO

Dato	Valor
Equipo	2 integrantes (Nicole y Santiago)
Tiempo	4 semanas · 12 h por semana por persona · **96 horas en total**
Sistema y editor	Windows + Visual Studio Code
Backend	Python (se puede elegir entre Python y Java)
Requisito del curso	La POO debe verse en el código; hay sustentación
Herramientas	HTML/CSS del prototipo + Flask + SQLite + PythonAnywhere (todas gratuitas)

Supuestos: el profesor evalúa la app publicada y la sustentación del código; tienen una cuenta de GitHub y no tienen tarjeta bancaria. Si alguno es distinto, hay que ajustar el plan.

1. Qué se hace y qué se recorta

Con 96 horas no caben todas las pantallas de la ficha. La app implementa el núcleo; el modelo de clases cubre toda la ficha.

Se queda	Se recorta
Registro e inicio de sesión de clientes	Proveedores y venta al por mayor
Salones y servicios	Profesionales independientes y domicilios reales
Reservar, cancelar y ver citas	Pagos (ya eran simulados)
Administrador básico	Merakié Pass y exportar .ics
Tienda con catálogo (opcional, semana 4)	Notificaciones

- Los descuentos quedan como una página estática, igual que en el prototipo.
- En `modelos.py` van **todas** las clases de la ficha: Usuario, Cliente, Administrador, TrabajadorIndependiente, Proveedor y ServicioBelleza.
- En la sustentación: «el modelo de clases cubre toda la ficha y la aplicación implementa el núcleo». Esto se explica en el informe.

2. Herramientas (una por capa)

Capa	Herramienta	Por qué es la más fácil para ustedes
Frontend	HTML + CSS de su prototipo, con plantillas de Flask (Jinja)	Ya lo saben. Reutilizan casi todo el HTML. No necesitan JavaScript nuevo.
Backend	Python + Flask	Una ruta es una función. Saben Python y POO. Java sin Spring exigiría mucha configuración (Maven o Gradle).
Base de datos	SQLite	Viene con Python (`sqlite3`). Es un solo archivo, sin servidor ni instalación.
Publicación	PythonAnywhere (plan gratis)	Soporta Flask y SQLite, y el archivo de datos se conserva. Normalmente no pide tarjeta.

> Antes de contar con PythonAnywhere, verifiquen en su sitio que el plan gratis siga permitiendo Flask y no pida tarjeta. Esas condiciones cambian. Se evitaron Render y Railway porque en sus planes gratis el archivo SQLite suele borrarse al reiniciar.

Qué hay que aprender

- **Flask:** rutas, `render_template`, formularios (`request.form`) y sesiones (`session`).
- **SQL básico:** CREATE TABLE, INSERT, SELECT, UPDATE, DELETE, JOIN.
- **Git básico:** add, commit, push.

Instalación en Windows, paso a paso

- Instalar **Python 3.12 o superior** desde python.org y marcar **«Add Python to PATH»**.
- Instalar **VS Code** con las extensiones *Python* y *SQLite Viewer*.
- Instalar **Git** desde git-scm.com y tener cuenta en GitHub.
- En la carpeta del proyecto, en la terminal de VS Code:

```
python -m venv venv
venv\Scripts\activate
pip install flask
```

Si PowerShell bloquea `activate`, ejecuten una vez: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`.

3. Organización del código

Se usan **clases para las entidades** y **funciones para las rutas** de Flask. Las clases muestran la POO de la ficha (herencia y el setter de edad); las rutas son funciones simples. No se usa ORM: el SQL se escribe a mano con `sqlite3`, así pueden explicar qué pasa con los datos en la sustentación.

```
merakie/
|-- app.py          <- rutas (funciones)
|-- modelos.py      <- clases Usuario, Cliente, Administrador, ...
|-- db.py           <- conexión y funciones SQL
|-- schema.sql      <- creación de tablas
|-- templates/      <- HTML del prototipo, dividido
`-- static/         <- CSS e imágenes
```

4. Diseño de la base de datos

Tabla	Columnas	Decisión
usuarios	id, nombre, documento, edad, correo (único), telefono, contrasena_hash, rol	Una sola tabla para Cliente y Administrador, separados por *rol*. Es más simple que una tabla por clase. La herencia sigue en el código: Cliente(Usuario) y Administrador(Usuario). Se guarda el hash, no la contraseña (`werkzeug.security`).
salones	id, nombre, zona, calificacion	Datos de cada establecimiento.
servicios	id, salon_id → salones, nombre, precio	Un salón tiene muchos servicios (1 a N), por eso el servicio lleva salon_id.
citas	id, usuario_id → usuarios, servicio_id → servicios, fecha, hora, modalidad, estado (activa/cancelada)	Une a un cliente con un servicio. Cancelar cambia *estado*, no borra: queda historial. El salón se obtiene por el servicio con un JOIN.
productos	id, nombre, descripcion, precio	Solo si hacen la tienda.

Relaciones: usuarios 1—N citas · salones 1—N servicios · servicios 1—N citas.

5. Módulos y por qué ese orden

Van del que no depende de nada hacia los que dependen de los anteriores. Primero publicar, luego leer datos, luego identificar al usuario, y al final lo que necesita usuario y datos juntos. Cada módulo funciona completo (pantalla + lógica + datos) y se publica al terminar.

Módulo	Contenido	Semana
M0	Publicar «Hola»	1
M1	Catálogo de salones (primera lectura de la base)	1
M2	Registro, login y clase Usuario	2
M3	Reservar, ver y cancelar citas	3
M4	Administrador básico	4
M5	Tienda (opcional)	4
Cierre	Pruebas, informe, diagrama de clases, ensayo de sustentación	4

6. Ruta semana a semana

Responsables: **N** = Nicole, **S** = Santiago. Cada tarea dura máximo 2 horas. Regla de trabajo: intercambien responsabilidades; quien escribe una ruta no debe ser quien revisa su SQL. Hagan *commit* al terminar cada tarea.

Semana 1

M0 · Hola publicado

Responsable	Tarea	Tiempo
N	Instalar Python, VS Code y Git; crear el venv	1.5 h
S	Crear el repo en GitHub y hacer el primer push	1 h
N	app.py con una ruta que devuelve «Hola»; correr en localhost:5000	1 h
S	Crear cuenta en PythonAnywhere, subir el código desde GitHub y configurar la web app	2 h
N+S	Abrir el link público en el celular	0.5 h

Debe funcionar al terminar: el link público muestra «Hola». Se comprueba abriéndolo desde un teléfono con datos móviles.

Error común: instalar Flask fuera del venv, o publicar sin recargar la web app. Pulsen **Reload** en PythonAnywhere tras cada `git pull`.

M1 · Catálogo de salones

Responsable	Tarea	Tiempo
S	Sacar el HTML de MERAKIE3.html (es un script de Python que lo genera) y dividirlo en base.html y salones.html	2 h
N	Escribir schema.sql con salones y servicios, y crear la base	2 h
S	Cargar los 4 salones del prototipo con INSERT y verlos en SQLite Viewer	1.5 h
N	Función obtener_salones() y ruta /salones	2 h
S	Mostrar salones y servicios con un bucle de Jinja {% for %}	2 h
N+S	Publicar	1 h

Debe funcionar al terminar: /salones lista los salones desde la base, no desde el HTML. Se comprueba cambiando un precio en SQLite Viewer y recargando.

Error común: guardar la base con una ruta relativa y que no se encuentre al publicar. Usen una ruta absoluta con `os.path`.

Semana 2

M2 · Registro, login y la clase Usuario

Responsable	Tarea	Tiempo
N	Clases Usuario, Cliente y Administrador en modelos.py, con el setter de edad 18–100	2 h
S	Tabla usuarios y función crear_usuario()	1.5 h
N	Formulario y ruta /registro, usando la clase para validar la edad	2 h
S	Ruta /login con session y hash de contraseña	2 h
N	Botón de cerrar sesión y nombre en el menú	1.5 h
S	Proteger rutas que requieren sesión	1.5 h
N+S	Publicar	1 h

Debe funcionar al terminar: registrarse con edad 17 es rechazado, con 25 funciona, y se puede entrar y salir. Se comprueba con ambas edades y entrando desde otro navegador.

Error común: guardar la contraseña tal cual, o dejar la SECRET_KEY vacía y que la sesión falle. Pongan una clave larga y no la suban a GitHub.

Semana 3

M3 · Reservar, ver y cancelar

Responsable	Tarea	Tiempo
S	Tabla citas y crear_cita()	1.5 h
N	Formulario de reserva con salón, servicio, fecha y hora	2 h
S	Función que calcula las horas libres consultando las citas existentes	2 h
N	Ruta /mis-citas con JOIN para mostrar salón y servicio	2 h
S	Cancelar cita cambiando estado	1.5 h
N	Mensajes de confirmación y de error	1.5 h
N+S	Publicar y probar con dos cuentas	1.5 h

Debe funcionar al terminar: una hora reservada por un usuario ya no aparece libre para otro, y cancelarla la libera. Se comprueba reservando con dos cuentas el mismo salón y hora.

Error común: calcular las horas libres sin descartar las citas canceladas. Filtren con `estado = 'activa'`.

Semana 4

M4 · Administrador básico

Responsable	Tarea	Tiempo
S	Crear un administrador a mano en la base	0.5 h
N	Ruta /admin que lista usuarios y todas las citas, solo si rol == 'administrador'	2 h
S	Agregar un salón y sus servicios desde un formulario	2 h

Debe funcionar al terminar: un cliente que entra a /admin es rechazado.

Error común: ocultar el botón pero no proteger la ruta. Hay que validar el rol en el servidor.

M5 · Tienda (opcional)

Solo si M4 queda lista a mitad de semana: tabla productos, catálogo y carrito guardado en session, sin pedidos persistentes (≈ 8 h).

Cierre (obligatorio)

Responsable	Tarea	Tiempo
N+S	Completar y documentar las clases sin pantalla (TrabajadorIndependiente, Proveedor, ServicioBelleza)	2 h
N+S	Pruebas de punta a punta con la app publicada	2 h
N+S	Informe y diagrama de clases con herencia (incluye la nota sobre el recorte)	3 h
N+S	Ensayo de sustentación: cada uno explica el módulo del otro	2 h

Con 24 horas por semana en equipo, el margen para imprevistos en la semana 4 es de unas pocas horas. Si algo se atrasa, la tienda (M5) es lo primero que se descarta.

7. Para la sustentación

- Cada integrante debe poder explicar **todo** el código, también el módulo que escribió el otro.
- Sugerencia: que cada uno cree su propia cuenta de GitHub, o al menos haga sus commits con su nombre. Así el historial muestra el aporte de los dos.
- Siguiente paso: pedir el **«Módulo 0»**. Se entrega el código completo con cada parte explicada, la subida a GitHub y la publicación en PythonAnywhere.